

Министерство образования Республики Беларусь
Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

Тема «Утилита для инвертирования слов»

Лабораторная работа №1
Интегрированная среда разработки по

Студент:
Преподаватель:

Н.С.Тисецкий
С.С. Силич

МИНСК 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Постановка задачи.....	4
1.1 Задание (вариант 8).....	4
1.2 Пример работы программы.....	4
2 Архитектура приложения (MVC)	5
2.1 Общее описание паттерна MVC.....	5
2.2 Активная и пассивная модель.....	5
2.3 Схема взаимодействия компонентов.....	5
3 Тестирование программы.....	6
3.1 Сценарии тестирования.....	6
3.2 Примеры работы программы.....	6
3.3 Результаты тестирования.....	7
ЗАКЛЮЧЕНИЕ.....	8
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	9
Приложение А.....	10

ВВЕДЕНИЕ

Цель работы изучить методы работы с одной из сред разработки (Visual Studio Code), научиться разрабатывать приложение с графическим интерфейсом на языке C++ с использованием библиотеки Qt, изучить паттерн MVC (Model-View-Controller) с активной моделью.

Задачи работы: Разработать утилиту для инвертирования слов в предложении. Реализовать графический интерфейс с главным окном и диалогом ввода. Реализовать запоминание последних введенных данных. Обеспечить обработку некорректных данных с выводом сообщения об ошибке. Применить архитектурный паттерн MVC с активной моделью. Используемые инструменты: Язык программирования: C++ (стандарт C++17) Библиотека: Qt 6.11.2 Компилятор: GCC 16.2.0 (MinGW-w64 UCRT64) Система сборки: qmake + mingw32-make Среда разработки: Visual Studio Code Окружение: MSYS2 UCRT64

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Задание (вариант 8)

Согласно заданию, необходимо разработать утилиту для инвертирования слов: по введённому предложению программа формирует предложение с инвертированными словами.

Требования к программе:

1. На главной форме должна быть кнопка «Ввести данные».
2. По нажатию на кнопку открывается дополнительное окно для ввода информации.
3. Последние введённые данные запоминаются в главной форме и восстанавливаются при повторном вводе.
4. После ввода на главной форме отображается результат — исходное и инвертированное предложения.
5. При вводе некорректных данных показывается сообщение об ошибке.
6. Реализовать каркас MVC с активной моделью.

1.2 Пример работы программы

Пример 1:

Ввод: привет мир

Результат: тевирип рим

Пример 2:

Ввод: программирование это интересно

Результат: еинавориммаргорп отэ онсеретни

Пример 3 (ошибка):

Ввод: пустая строка

Результат: сообщение об ошибке «Введены некорректные данные! Пожалуйста, введите непустое предложение.»

2 АРХИТЕКТУРА ПРИЛОЖЕНИЯ (MVC)

2.1 Общее описание паттерна MVC

Паттерн Model-View-Controller (MVC) — это архитектурный шаблон проектирования, разделяющий приложение на три взаимосвязанных компонента:

Model (Модель) — хранит данные и бизнес-логику приложения. Не зависит от интерфейса.

View (Представление) — отображает данные пользователю и обрабатывает пользовательский ввод.

Controller (Контроллер) — связывает Model и View, обрабатывает действия пользователя и обновляет модель.

2.2 Активная и пассивная модель

В данном приложении реализована активная модель. Это означает, что:

Модель самостоятельно уведомляет об изменениях через механизм сигналов и слотов Qt.

View подписывается на сигналы модели и автоматически обновляется.

Контроллер выполняет роль посредника, но не управляет обновлением View напрямую.

Сигналы активной модели:

`dataChanged()` — испускается при успешном изменении данных.

`errorOccurred(const QString&)` — испускается при возникновении ошибки.

2.3 Схема взаимодействия компонентов

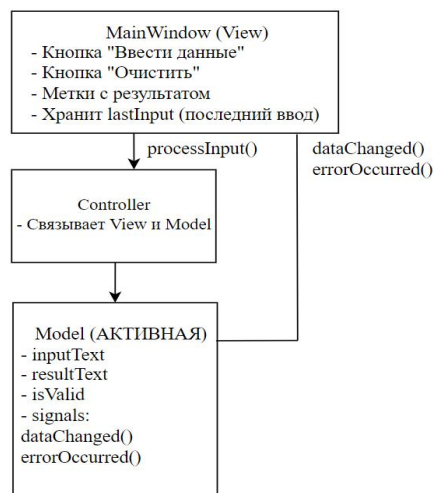


Рисунок 2.1 — Схема взаимодействия компонентов MVC

3 ТЕСТИРОВАНИЕ ПРОГРАММЫ

3.1 Сценарии тестирования

Таблица 3.1 – Сценарии тестирования

№	Сценарий	Ожидаемый результат
1	Ввод «привет мир»	«тевирп рим»
2	Ввод «программирование это интересно»	«еинавориммаргорп онсеретни»
3	Ввод пустой строки	Сообщение об ошибке
4	Повторное открытие диалога	Восстановление последнего ввода
5	Нажатие «Очистить»	Сброс отображения
6	Ввод одного слова	Корректное инвертирование

3.2 Примеры работы программы

Тест 1: Инвертирование двух слов

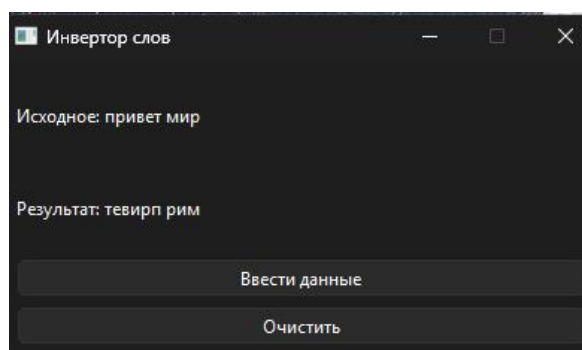


Рисунок 3.1 — Результат инвертирования предложения «привет мир»

Тест 2: Инвертирование длинного предложения

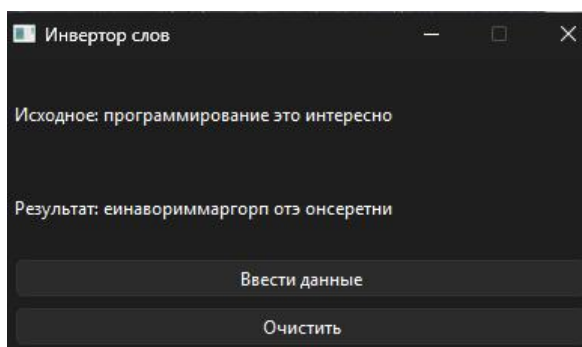


Рисунок 3.2 — Результат инвертирования длинного предложения

Тест 3: Обработка ошибки

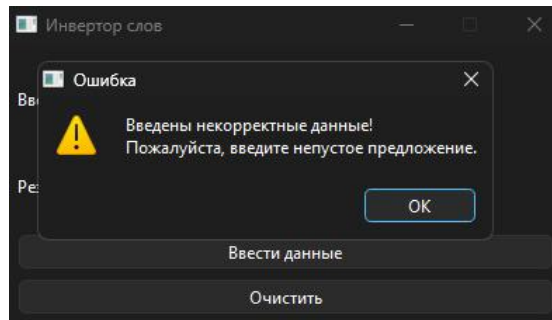


Рисунок 3.3 — Сообщение об ошибке при некорректных данных

Тест 4: Восстановление последнего ввода

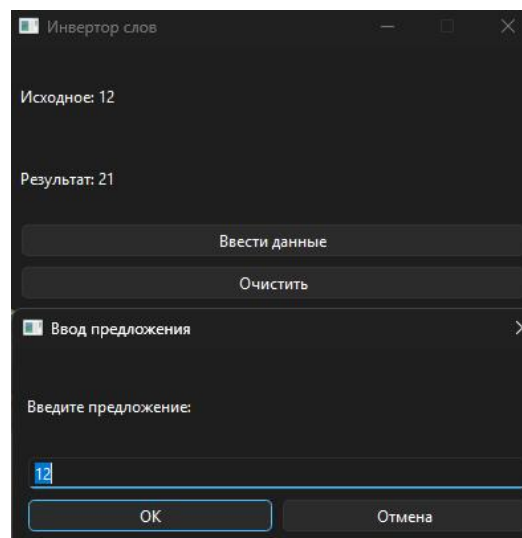


Рисунок 3.4 — Восстановление последнего введённого текста

3.3 Результаты тестирования

Все тесты пройдены успешно. Программа корректно:

1. Инвертирует слова в предложении.
2. Обрабатывает пустые строки с выводом сообщения об ошибке.
3. Запоминает последний ввод и восстанавливает его при повторном открытии диалога.
4. Сбрасывает отображение при нажатии кнопки «Очистить».

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы №1 была разработана утилита для инвертирования слов на языке C++ с использованием библиотеки Qt 6.11.2. Изучена среда разработки Visual Studio Code и окружение MSYS2 UCRT64. Освоена работа с библиотекой Qt: создание графического интерфейса, сигналы и слоты, диалоговые окна. Реализован архитектурный паттерн MVC с активной моделью: Модель (Model) хранит данные и испускает сигналы `dataChanged()` и `errorOccurred()`. Представление (MainWindow, InputDialog) отображает данные и подписывается на сигналы модели. Контроллер (Controller) связывает модель и представление. Реализовано запоминание последнего ввода и его восстановление. Реализована обработка некорректных данных с выводом сообщения об ошибке. Проведено тестирование, подтвердившее корректность работы программы. Цель лабораторной работы достигнута, все задачи выполнены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Официальная документация Qt 6 [Электронный ресурс]. — Режим доступа: <https://doc.qt.io/qt-6/>. — Дата доступа: 14.09.2026.
- [2] Шлее, М. Qt 6. Разработка оконных приложений на C++ / М. Шлее. — СПб. : БХВ-Петербург, 2023. — 640 с.
- [3] Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. — 10-е изд. — М. : Вильямс, 2020. — 896 с.
- [4] Документация MSYS2 [Электронный ресурс]. — Режим доступа: <https://www.msys2.org/docs/>. — Дата доступа: 14.09.2026.
- [5] СТП 01–2017. Дипломные проекты (работы). Общие требования. — Минск : БГУИР, 2017. — 168 с.

ПРИЛОЖЕНИЕ А

(обязательное)

Текст программы

```
1  //Model
2  #ifndef MODEL_H
3  #define MODEL_H
4
5  #include <QObject>
6  #include <QString>
7  #include <QStringList>
8
9  class Model : public QObject {
10      Q_OBJECT
11
12  private:
13      QString inputText;          // исходное предложение
14      QString resultText;        // инвертированное предложение
15      bool isValid;              // флаг корректности данных
16
17  public:
18      explicit Model(QObject* parent = nullptr);
19
20      void setInputText(const QString& text);
21      QString getInputText() const;
22      QString getResultText() const;
23      bool isValidData() const;
24
25  signals:
26      void dataChanged();          // данные изменились
27      void errorOccurred(const QString& message); // ошибка
28
29  private:
30      void processText();          // обработка текста
31      QString invertWord(const QString& word); // инвертирование слова
32  };
33 #endif // MODEL_H
```

```

34 //Controller
35 #ifndef CONTROLLER_H
36 #define CONTROLLER_H
37
38 #include <QObject>
39 #include "Model.h"
40
41 class Controller : public QObject {
42     Q_OBJECT
43
44 private:
45     Model* model;
46
47 public:
48     explicit Controller(QObject* parent = nullptr);
49     ~Controller();
50
51     Model* getModel() const;
52     void processInput(const QString& text);
53     void clearData();
54     QString getInputText() const;
55     QString getResultText() const;
56     bool isValidData() const;
57 };
58
59 #endif // CONTROLLER_H
60 //MainWindow
61 #ifndef MAINWINDOW_H
62 #define MAINWINDOW_H
63
64 #include <QMainWindow>
65 #include <QLabel>
66 #include <QPushButton>
67 #include "Controller.h"
68 #include "InputDialog.h"
69
70 class MainWindow : public QMainWindow {
71     Q_OBJECT

```

```

72
73 private:
74     Controller* controller;
75     QLabel* inputLabel;
76     QLabel* resultLabel;
77     QPushButton* inputButton;
78     QPushButton* clearButton;
79     QString lastInput;    // запоминание последнего ввода
80
81 public:
82     MainWindow(QWidget* parent = nullptr);
83     ~MainWindow();
84
85 private slots:
86     void onInputButtonClicked();
87     void onClearButtonClicked();
88     void onDataChanged();
89     void onErrorOccurred(const QString& message);
90     void updateDisplay();
91 };
92 #endif // MAINWINDOW_H
93 //InputDialog
94 #ifndef INPUTDIALOG_H
95 #define INPUTDIALOG_H
96
97 #include <QDialog>
98 #include <QLabel>
99 #include <QLineEdit>
100 #include <QPushButton>
101 #include <QVBoxLayout>
102 #include <QHBoxLayout>
103
104 class InputDialog : public QDialog {
105     Q_OBJECT
106
107 private:
108     QLineEdit* inputLineEdit;
109     QPushButton* okButton;

```

```

110     QPushButton* cancelButton;
111
112 public:
113     InputDialog(const QString& initialText = "", QWidget* parent =
nullptr);
114     QString getInputText() const;
115
116 private slots:
117     void onOkClicked();
118 };
119 #endif // INPUTDIALOG_H
120 //L1.pro
121 QT += widgets core
122
123 TARGET = L1
124 TEMPLATE = app
125
126 SOURCES += \
127     main.cpp \
128     MainWindow.cpp \
129     InputDialog.cpp \
130     Model.cpp \
131     Controller.cpp
132
133 HEADERS += \
134     MainWindow.h \
135     InputDialog.h \
136     Model.h \
137     Controller.h
138
139 CONFIG += c++17
140 //main.cpp
141 #include <QApplication>
142 #include "MainWindow.h"
143
144 int main(int argc, char *argv[])
145 {
146     QApplication app(argc, argv);

```

```

147     MainWindow window;
148     window.show();
149     return app.exec();
150 }
151 //Model.cpp
152 #include "Model.h"
153
154 Model::Model(QObject* parent)
155     : QObject(parent), isValid(false) {}
156
157 void Model::setInputText(const QString& text) {
158     inputText = text;
159     isValid = !text.trimmed().isEmpty();
160
161     if (isValid) {
162         processText();
163         emit dataChanged(); // уведомляем View
164     } else {
165         resultText = "";
166         emit errorOccurred("Введены некорректные данные!\n"
167                             "Пожалуйста, введите непустое предложение.");
168     }
169 }
170 QString Model::getInputText() const {
171     return inputText;
172 }
173 QString Model::getResultText() const {
174     return resultText;
175 }
176 bool Model::isValidData() const {
177     return isValid;
178 }
179 void Model::processText() {
180     if (!isValid) return;
181
182     QStringList words = inputText.split(' ', Qt::SkipEmptyParts);
183     QStringList invertedWords;
184

```

```

185     for (const QString& word : words) {
186         invertedWords.append(invertWord(word));
187     }
188     resultText = invertedWords.join(' ');
189 }
190 QString Model::invertWord(const QString& word) {
191     QString inverted;
192     for (int i = word.length() - 1; i >= 0; --i) {
193         inverted.append(word[i]);
194     }
195     return inverted;
196 }
197 //Controller.cpp
198 #include "Controller.h"
199
200 Controller::Controller(QObject* parent)
201     : QObject(parent) {
202     model = new Model(this);
203 }
204
205 Controller::~~Controller() {}
206
207 Model* Controller::getModel() const {
208     return model;
209 }
210 void Controller::processInput(const QString& text) {
211     model->setInputText(text);
212 }
213 void Controller::clearData() {
214     model->setInputText("");
215 }
216 QString Controller::getInputText() const {
217     return model->getInputText();
218 }
219 QString Controller::getResultText() const {
220     return model->getResultText();
221 }
222 bool Controller::isValidData() const {

```

```

223     return model->isValidData();
224 }
225 //MainWindow.cpp
226 #include "MainWindow.h"
227 #include <QMessageBox>
228 #include <QVBoxLayout>
229 #include <QWidget>
230
231 MainWindow::MainWindow(QWidget* parent)
232     : QMainWindow(parent), lastInput("") {
233
234     controller = new Controller(this);
235
236     // Подключение к сигналам активной модели
237     connect(controller->getModel(), &Model::dataChanged,
238             this, &MainWindow::onDataChanged);
239     connect(controller->getModel(), &Model::errorOccurred,
240             this, &MainWindow::onErrorOccurred);
241
242     // Создание виджетов
243     inputLabel = new QLabel("Введите предложение:", this);
244     resultLabel = new QLabel("Результат: ", this);
245     inputButton = new QPushButton("Ввести данные", this);
246     clearButton = new QPushButton("Очистить", this);
247
248     // Layout
249     QVBoxLayout* layout = new QVBoxLayout();
250     layout->addWidget(inputLabel);
251     layout->addWidget(resultLabel);
252     layout->addWidget(inputButton);
253     layout->addWidget(clearButton);
254
255     QWidget* centralWidget = new QWidget(this);
256     centralWidget->setLayout(layout);
257     setCentralWidget(centralWidget);
258
259     // Подключение кнопок
260     connect(inputButton, &QPushButton::clicked,

```

```

261         this, &MainWindow::onInputButtonClicked);
262     connect(clearButton, &QPushButton::clicked,
263         this, &MainWindow::onClearButtonClicked);
264
265     setFixedSize(400, 200);
266     setWindowTitle("Инвертор слов");
267 }
268 MainWindow::~MainWindow() {}
269
270 void MainWindow::onInputButtonClicked() {
271     InputDialog dialog(lastInput, this);
272
273     if (dialog.exec() == QDialog::Accepted) {
274         QString input = dialog.getInputText();
275         lastInput = input;           // запоминаем последний ввод
276         controller->processInput(input);
277     }
278 }
279 void MainWindow::onClearButtonClicked() {
280     lastInput = "";
281     controller->clearData();
282     inputLabel->setText("Введите предложение:");
283     resultLabel->setText("Результат: ");
284 }
285 void MainWindow::onDataChanged() {
286     updateDisplay();
287 }
288 void MainWindow::onErrorOccurred(const QString& message) {
289     QMessageBox::warning(this, "Ошибка", message);
290 }
291 void MainWindow::updateDisplay() {
292     if (controller->isValidData()) {
293         inputLabel->setText("Исходное: " + controller->getInputText());
294         resultLabel->setText("Результат: " + controller->getResultText());
295     }
296 }
297 //INPUTDIALOG.CPP

```

```

298 #include "InputDialog.h"
299
300 InputDialog::InputDialog(const QString& initialText, QWidget* parent)
301     : QDialog(parent) {
302
303     setWindowTitle("Ввод предложения");
304     setFixedSize(400, 150);
305
306     QLabel* label = new QLabel("Введите предложение:", this);
307     inputLineEdit = new QLineEdit(this);
308     inputLineEdit->setText(initialText);    // восстановление последнего
ввода
309     inputLineEdit->selectAll();
310
311     okButton = new QPushButton("OK", this);
312     cancelButton = new QPushButton("Отмена", this);
313
314     QVBoxLayout* mainLayout = new QVBoxLayout();
315     mainLayout->addWidget(label);
316     mainLayout->addWidget(inputLineEdit);
317
318     QHBoxLayout* buttonLayout = new QHBoxLayout();
319     buttonLayout->addWidget(okButton);
320     buttonLayout->addWidget(cancelButton);
321     mainLayout->addLayout(buttonLayout);
322
323     setLayout(mainLayout);
324
325     connect(okButton, &QPushButton::clicked,
326             this, &InputDialog::onOkClicked);
327     connect(cancelButton, &QPushButton::clicked,
328             this, &QDialog::reject);
329     connect(inputLineEdit, &QLineEdit::returnPressed,
330             this, &InputDialog::onOkClicked);
331 }
332
333 QString InputDialog::getInputText() const {
334     return inputLineEdit->text();
335 }

```

```
335 void InputDialog::onOkClicked() {  
336     accept();  
337 }
```